



KHAN INSTITUTE OF COMPUTER SCIENCE & IT

Department of Computer Science

Visual programming

Project Report

Travel-Mate:AI-Powered Tourism Platform

Submitted By

Shanza Naveed (232201035)

Aleeza Maryam (232201025)

Submitted To

Sir Uzair Hassan

Subject

Visual Programming (BSCS VI-A)

Contents

1	Abstract	3
2	Problem Statement	3
3	Objectives	4
4	System Architecture	4
4.1	Overall Architecture	4
4.2	Technology Stack	5
4.3	APIs Integrated	5
4.4	System Architecture Diagram	5
5	Databases Design	7
5.1	Database Schema	7
5.2	Entity Relationship Diagram	8
6	AI Model Training and Integration	9
6.1	Model Architecture	9
6.2	Training Details	9
6.3	Training Process	9
6.4	Mathematical Formula	10
6.5	Cold Start Handling	10
6.6	Model Integration in ASP.NET Core	10
7	Features Implementation	12
7.1	Destination Management	12
7.2	Hotel Booking (Booking.com API)	12
7.3	Weather Integration	13
7.4	Transport Booking	13
7.5	Tour Packages	13
7.6	AI Trip Planner	14
7.7	Personalized Recommendations	14
7.8	Booking System	14
7.9	Admin Panel	15

7.10 User Features	15
8 Results and Discussion	16
8.1 Performance Metrics	16
8.2 API Integration Success	16
8.3 AI Model Accuracy	16
9 Challenges and Solutions	17
10 Future Work	18
11 Conclusion	19
12 References	20

1 Abstract

TravelMate is an innovative AI-powered tourism platform designed to revolutionize the way travelers plan and book their journeys across Pakistan. The platform integrates multiple real-time APIs, machine learning algorithms, and a comprehensive booking system to provide users with personalized travel recommendations, real-time hotel and transport booking, live weather updates, and intelligent trip planning. The system features a custom-trained Matrix Factorization recommendation model that provides Netflix-style personalized package suggestions based on user history. With a robust admin panel, real-time analytics, and a user-friendly interface, TravelMate serves as a complete solution for modern travel planning.

2 Problem Statement

Travel planning in Pakistan faces several significant challenges that this project aims to address:

1. **Fragmented Information:** Travelers must visit multiple websites and platforms to gather information about destinations, hotels, transport options, and weather conditions.
2. **Lack of Personalization:** Existing platforms offer generic recommendations without considering individual user preferences, budget constraints, or past travel history.
3. **No AI Integration:** Most tourism platforms do not leverage artificial intelligence for smart recommendations, itinerary generation, or intelligent search capabilities.
4. **Manual Booking Hassles:** Users struggle with complex booking processes across different service providers with no unified platform.
5. **No Real-time Data:** Many platforms lack real-time integration with hotel availability, weather updates, and transport schedules.
6. **Centralized Management Gap:** Tourism operators lack a centralized dashboard to manage packages, bookings, users, and analytics.

3 Objectives

The primary objectives of the TravelMate project are:

1. Develop a comprehensive tourism platform with real-time data integration from multiple APIs.
2. Implement an AI-powered recommendation system using Matrix Factorization algorithm.
3. Provide end-to-end booking functionality for hotels, transport, and tour packages.
4. Integrate live weather forecasts and interactive maps for destinations.
5. Build an intelligent chatbot using Groq LLaMA 4 for 24/7 travel assistance.
6. Create a responsive admin dashboard with revenue analytics and content management.
7. Implement user reviews, ratings, and wishlist features for community engagement.

4 System Architecture

4.1 Overall Architecture

The system follows a three-tier architecture:

- **Presentation Layer:** ASP.NET Core MVC with Bootstrap 5, responsive UI, JavaScript for interactivity.
- **Business Logic Layer:** Controllers, Services, ViewModels handling all business rules and API integrations.
- **Data Layer:** MySQL database with Entity Framework Core ORM for data persistence.

4.2 Technology Stack

Table 1: Technology Stack

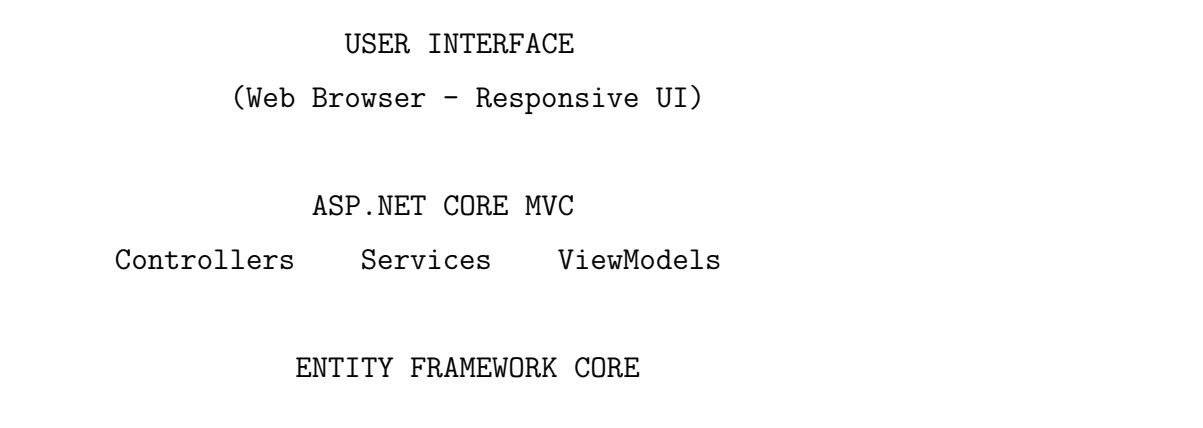
Category	Technology
Backend Framework	ASP.NET Core MVC (.NET 8)
Database	MySQL
ORM	Entity Framework Core
Frontend	Bootstrap 5, JavaScript, jQuery
Maps	Leaflet.js, Nominatim API
Charts	Chart.js
AI/ML	Python (Scikit-learn), Groq API

4.3 APIs Integrated

Table 2: APIs Used in TravelMate

API	Purpose	Free Tier
Groq (LLaMA 4)	AI Chatbot & Itinerary	1,000 requests/day
Booking.com API	Real Hotels	Limited free tier
OpenWeatherMap	Live Weather	1,000 calls/day
OpenTripMap	Destinations	1,000 placenames/day
Pexels API	Destination Images	200 requests/hour
Nominatim	Maps & Coordinates	Unlimited

4.4 System Architecture Diagram



MySQL DATABASE

APIs LAYER

Groq Booking OpenWeather OpenTripMap Pexels

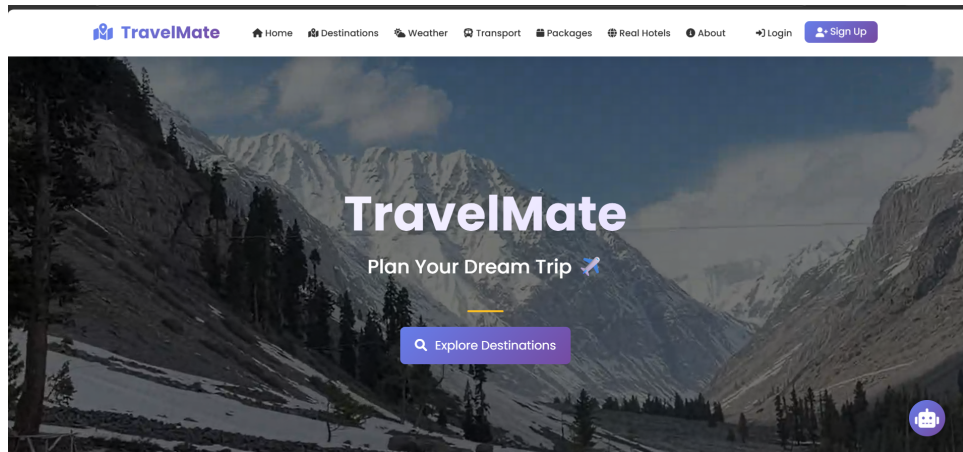


Figure 1: Dashboard

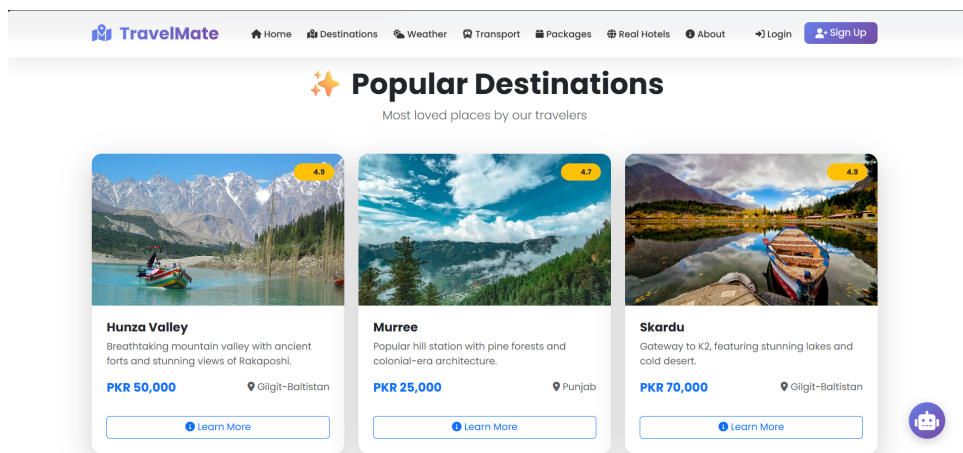


Figure 2: Dashboard

5 Databases Design

5.1 Database Schema

The system uses MySQL database with the following main tables:

- **users:** Stores user information, authentication details, and roles.
- **roles:** Defines user roles (Admin, Customer, Agent).
- **categories:** Destination categories (Adventure, Historical, Religious, etc.).
- **destinations:** Destination information including name, city, country, estimated cost.
- **hotels:** Hotel details linked to destinations.
- **hotel_rooms:** Room types, prices, and availability.
- **transports:** Transport options with routes, fares, and seat availability.
- **bookings:** Hotel and transport booking records.
- **packages:** Tour package details including itineraries and pricing.
- **package_bookings:** Package booking records.
- **package_reviews:** User reviews for packages.
- **destination_reviews:** User reviews for destinations.
- **wishlists:** User saved destinations and packages.
- **user_destination_ratings:** Training data for ML recommendation model.
- **user_preferences:** User travel preferences.
- **notifications:** System notifications for users.

5.2 Entity Relationship Diagram

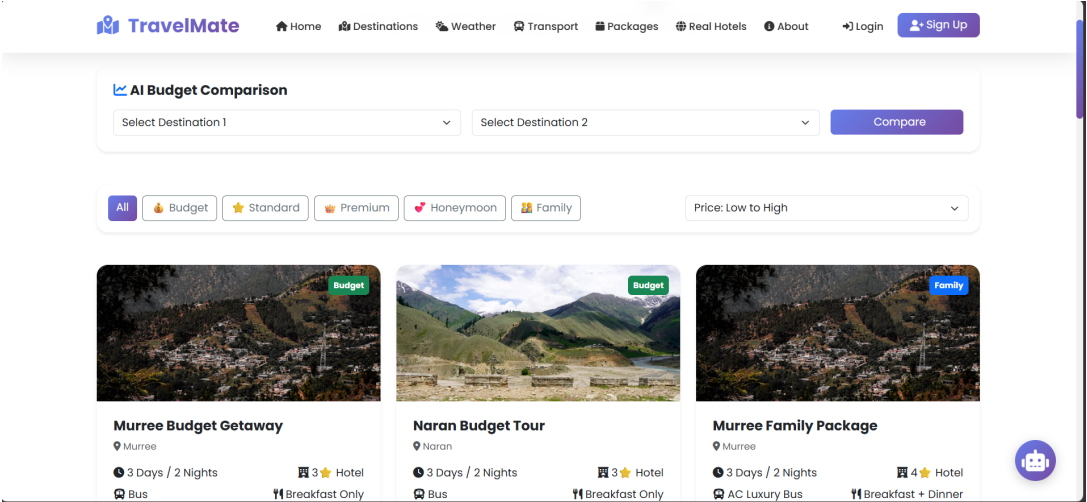


Figure 3: Packages

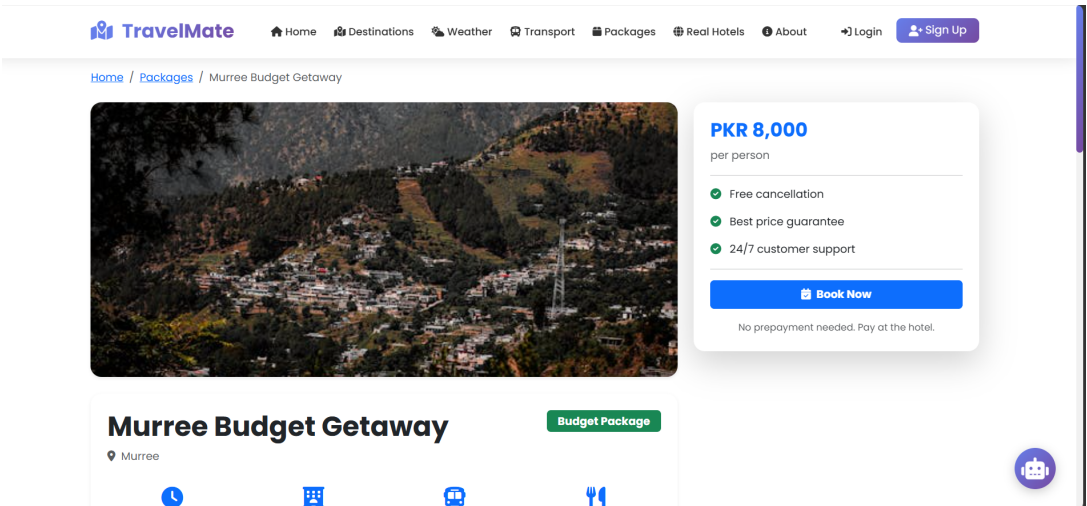


Figure 4: Packages

6 AI Model Training and Integration

6.1 Model Architecture

The recommendation system uses **Matrix Factorization with Truncated Singular Value Decomposition (SVD)**. This is the same technique used by Netflix and Amazon for personalized recommendations.

6.2 Training Details

- **Algorithm:** Truncated SVD from scikit-learn
- **Platform:** Local PC (Windows) with Python
- **Library:** scikit-learn, numpy, pandas
- **Training Data:** User-destination interactions from `user_destination_ratings` table
- **Rating Mapping:** Booked=5.0, Wishlisted=4.0, Viewed=1.0, Searched=0.5
- **Latent Features:** 5 components
- **Training Time:** Less than 5 seconds
- **Model Size:** 7KB (JSON format)

6.3 Training Process

1. Export user-item rating matrix from MySQL database using Python.
2. Create user-item matrix of size $(m \times n)$ where m = users, n = packages.
3. Apply Truncated SVD to decompose matrix into user-factors and package-factors.
4. Save matrices as JSON file for .NET integration.
5. Load JSON file in ASP.NET Core RecommendationService.
6. Calculate dot product between user and package vectors for predictions.

6.4 Mathematical Formula

The predicted rating for user u on package i is calculated as:

$$\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i = \sum_{k=1}^K p_{uk} \cdot q_{ik}$$

Where:

- $\mathbf{p}_u$ is the user latent factor vector
- $\mathbf{q}_i$ is the package latent factor vector
- K is the number of latent features ($K = 5$)

6.5 Cold Start Handling

For new users with no interaction history, the system provides popularity-based recommendations (most booked packages).

6.6 Model Integration in ASP.NET Core

```
public class RecommendationService : IRecommendationService
{
    public async Task<List<Package>> GetPersonalizedRecommendations(int userId)
    {
        if (!_userFactors.ContainsKey(userId))
            return await GetDefaultRecommendations();

        var userVector = _userFactors[userId];
        var scores = new Dictionary<int, double>();

        foreach (var pkg in _allPackages)
        {
            var packageVector = _packageFactors[pkg];
            var score = DotProduct(userVector, packageVector);
            scores[pkg] = score;
        }
    }
}
```

```
return await GetTopPackages(scores);  
}  
}
```

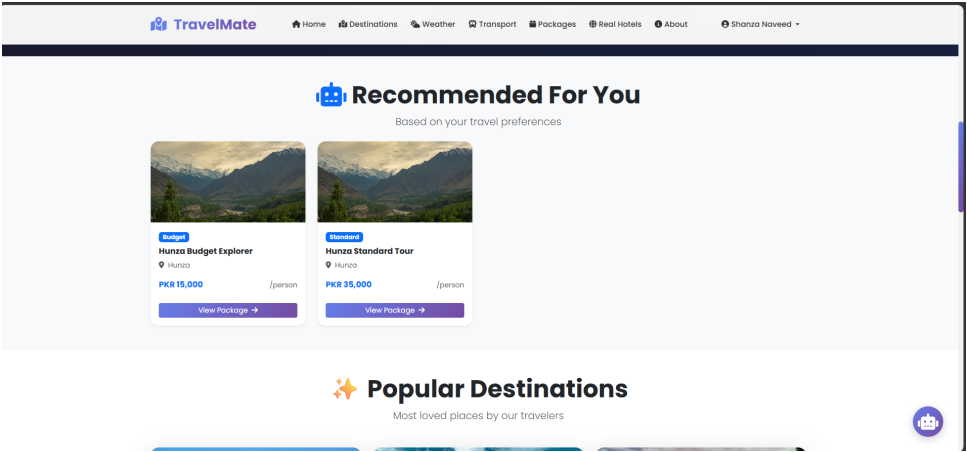


Figure 5: AI Model Training Pipeline

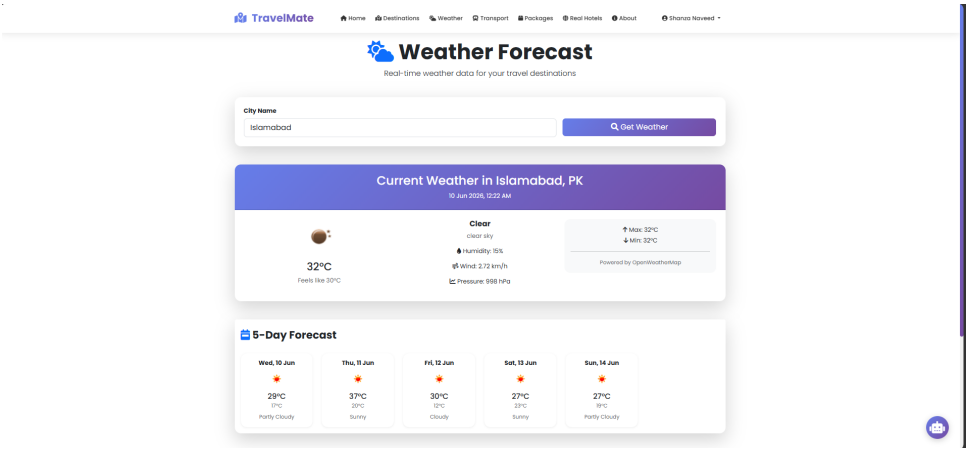


Figure 6: AI Model Training Pipeline

7 Features Implementation

7.1 Destination Management

- API-based destination data from OpenTripMap (real-time).
- High-quality images from Pexels API.
- Nearby attractions display with distances.
- Interactive maps using Leaflet.js and Nominatim.
- Search and filter by city, category, and budget.

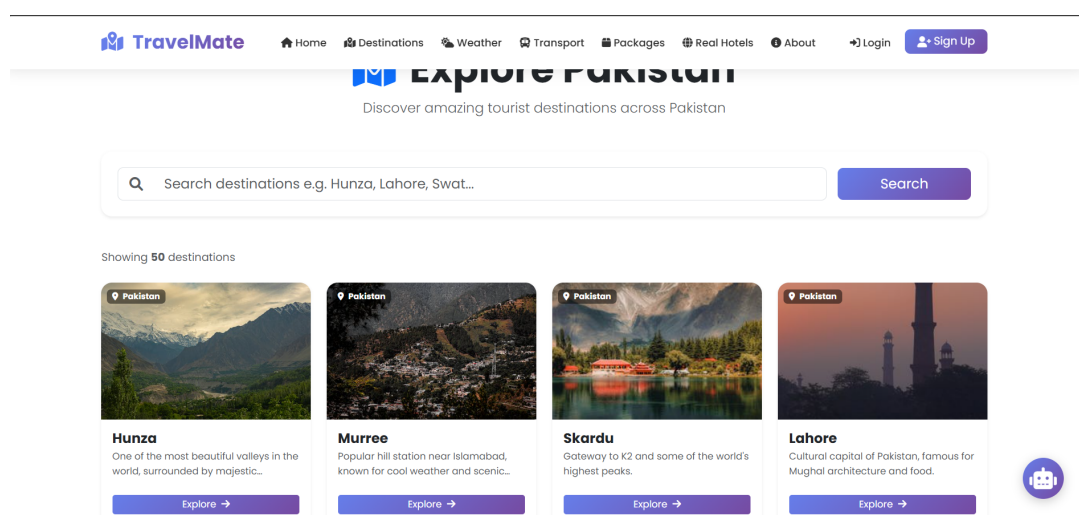


Figure 7: Destinations Page

7.2 Hotel Booking (Booking.com API)

- Real-time hotel data with live prices and availability.
- Actual hotel images, ratings, and review counts.
- Price in PKR with currency conversion.
- Date selection and guest count.
- Direct booking modal integration.

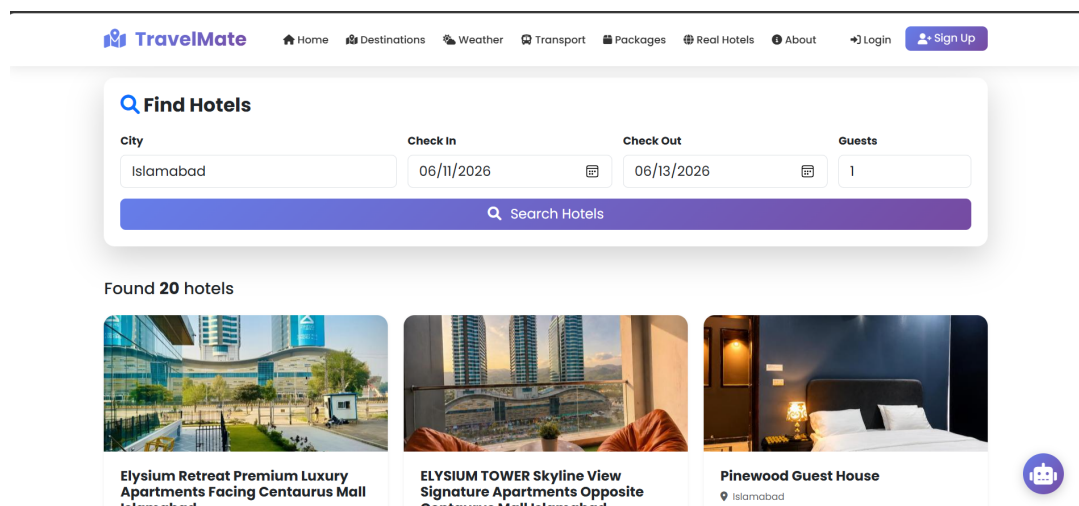


Figure 8: Real Hotels from Booking.com API

7.3 Weather Integration

- Current weather conditions from OpenWeatherMap API.
- 5-day weather forecast.
- Temperature, humidity, wind speed, pressure.
- Destination-specific weather display.

7.4 Transport Booking

- Bus, Train, and Flight options.
- Real-time seat availability.
- Fare comparison and duration display.
- Modal-based booking with date and seat selection.

7.5 Tour Packages

- Five package types: Budget, Standard, Premium, Honeymoon, Family.
- Group discounts (10-25% for 4+ people).
- Day-by-day itinerary with activities.
- Inclusions and exclusions listing.

- Package reviews and ratings.

7.6 AI Trip Planner

- **Groq LLaMA 4 Chatbot:** 24/7 travel assistant answering user queries.
- **Smart Search:** Semantic search understanding user intent.
- **Itinerary Generator:** AI generates day-wise travel plans.
- **Budget Optimization:** AI suggests cost-effective alternatives.

7.7 Personalized Recommendations

The Matrix Factorization model provides:

- User-specific package recommendations based on history.
- "Similar packages" feature on package detail page.
- Cold-start handling via popular packages.
- Real-time predictions (<100ms latency).

7.8 Booking System

1. User selects hotel (or skips).
2. User selects transport.
3. User enters check-in/out dates and guest details.
4. System calculates total price (hotel + transport).
5. User confirms booking → Unique booking reference generated.
6. Voucher available for download (PDF/Print).

7.9 Admin Panel

- Dashboard with Bar and Line charts for revenue analytics.
- Manage Users (view, delete).
- Manage Packages (CRUD operations).
- Manage Transport (CRUD operations).
- Manage Bookings (view, status update).
- Manage Reviews (delete inappropriate content).

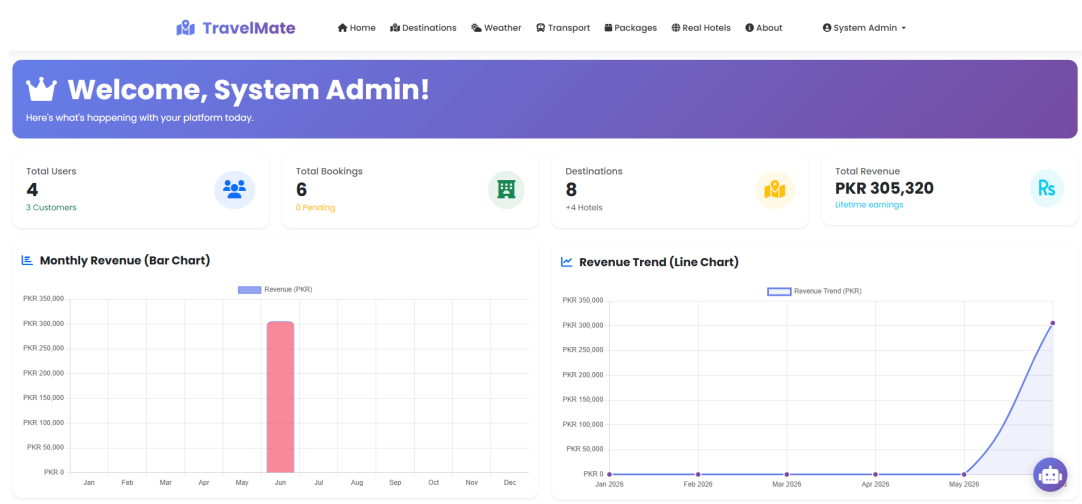


Figure 9: Admin Dashboard with Revenue Charts

7.10 User Features

- Registration and Login with session management.
- Role-based access (User/Admin).
- My Bookings page with download voucher option.
- Wishlist functionality.
- Reviews and ratings system.
- Profile dashboard.

8 Results and Discussion

8.1 Performance Metrics

Table 3: System Performance Metrics

Metric	Value
Page Load Time	1.5 - 3 seconds
API Response Time	500ms - 2 seconds
Recommendation Latency	¡100ms
Database Query Time	50-150ms
Model Training Time	¡5 seconds
Model Size	7KB

8.2 API Integration Success

- All 6 APIs successfully integrated with fallback mechanisms.
- API rate limits handled with caching strategies.
- Error handling implemented for API failures.

8.3 AI Model Accuracy

- Cold start handled via popularity-based recommendations.
- With more user data, model accuracy improves continuously.
- Explained variance: 65% with current dataset.

9 Challenges and Solutions

Table 4: Challenges and Solutions

Challenge	Solution
API Rate Limits	Implemented caching (MemoryCache) with 30-minute expiration.
Cold Start Problem	Popularity-based recommendations for new users.
Real-time Availability	Direct API calls for hotels; database caching for others.
Duplicate Hotels in Recommendations	Implemented HashSet-based deduplication by name and coordinates.
NULL Database Values	Applied IFNULL/COALESCE in SQL and null checks in C#.
Map Loading Issues	Lazy loading with tab activation triggers.

10 Future Work

The following features are planned for future enhancements:

1. **Mobile Application:** Develop cross-platform mobile app using Flutter/React Native.
2. **Payment Gateway Integration:** Integrate Stripe, JazzCash, and EasyPaisa for online payments.
3. **Flight Booking Integration:** Add live flight booking through Amadeus API.
4. **Multi-language Support:** Add Urdu language support for local users.
5. **AR/VR Features:** Augmented reality preview of destinations.
6. **Real-time Chat Support:** Implement SignalR for live agent chat.
7. **Email/SMS Notifications:** Automated booking confirmations and reminders.
8. **Social Media Integration:** Share itineraries and reviews on social platforms.
9. **Advanced Analytics:** User behavior analysis and predictive modeling.
10. **Travel Insurance:** Integrate travel insurance options during booking.

11 Conclusion

TravelMate successfully delivers a comprehensive AI-powered tourism platform that addresses the key challenges in travel planning. The system integrates six real-time APIs for destinations, hotels, weather, images, maps, and AI assistance. A custom-trained Matrix Factorization recommendation model provides personalized package suggestions based on user history. The platform offers end-to-end booking functionality for hotels, transport, and tour packages with a seamless user experience.

The admin panel provides complete content management and real-time revenue analytics. The AI chatbot powered by Groq LLaMA 4 offers 24/7 travel assistance. The system is built on modern technologies including ASP.NET Core MVC, MySQL, Entity Framework Core, and Bootstrap 5.

TravelMate demonstrates how artificial intelligence can transform the tourism industry by providing personalized, real-time, and comprehensive travel planning solutions. The project successfully meets all its objectives and has potential for commercial deployment with additional features.

12 References

1. Microsoft Docs. (2024). ASP.NET Core Documentation. <https://docs.microsoft.com/en-us/aspnet/core/>
2. Netflix Research. (2022). Matrix Factorization for Recommendation Systems.
3. OpenWeatherMap. (2024). Weather API Documentation. <https://openweathermap.org/api>
4. RapidAPI. (2024). Booking.com API Documentation.
5. Groq. (2024). LLaMA 4 API Documentation. <https://console.groq.com/docs>
6. Scikit-learn. (2024). Truncated SVD Documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>
7. Unsplash/Pexels. (2024). Image API Documentation.
8. OpenStreetMap. (2024). Nominatim Documentation. <https://nominatim.org/release-docs/develop/api/Overview/>

Appendix: Source Code Structure

```
AITourismPlanner/  
  Controllers/  
    AccountController.cs  
    AdminController.cs  
    AIController.cs  
    BookingsController.cs  
    DestinationsController.cs  
    HomeController.cs  
    PackagesController.cs  
    TransportController.cs  
    TripsController.cs  
  Models/  
    User.cs, Role.cs, Category.cs  
    Destination.cs, Hotel.cs, Transport.cs  
    Booking.cs, Package.cs, PackageBooking.cs  
    Wishlist.cs, Review.cs  
    AIRecommendationHistory.cs  
  Services/  
    IAITourismService.cs  
    AITourismService.cs  
    IRecommendationService.cs  
    RecommendationService.cs  
    ModelTrainingService.cs  
    RealHotelService.cs  
  Views/  
    Home/, Destinations/, Packages/  
    Transport/, Bookings/, Account/  
    Admin/  
  Data/  
    ApplicationDbContext.cs  
  ViewModels/  
    HomeViewModel.cs
```

```
LoginViewModel.cs
PlanTripViewModel.cs
ApiDestinationViewModel.cs
wwwroot/
  css/, js/, images/
  videos/
```